PETSURE · 90STP

STP90 Architecture Vision

Four tiers of change for Horizon 1, the motivation behind each gateway, and a target for medical assessment.

Danilo · Architecture · October 2026 · Draft for review

The North Star: our Horizon 2 target

Platform

A simpler, modern platform that is easier and safer to run and change.

Reached through tiers 1 to 3: modernisation, plumbing and data integrity.

Medical assessment

Medical assessment designed so that AI can be plugged in.

Reached through tier 4: expanding STP % and ML integration.

Vision (BMM): a headless, AI-first claims platform that automates the customer journey across multiple insurers, with humans in the loop.

Four tiers of change

Each tier brings us closer to the North Star and prepares PetSure for the next generation of AI technology.

Tier 1

Modernisation

- Lift and shift to Docker
- Blue / green deployments
- Multi-insurance environments
- Catch 1, 2 and 3

Tier 2

Refactor the plumbing between components

- Decouple Clawdia from UPM
- VetHub / COS polling to events

Tier 3

Fix data integrity at the source

- Duplicated Service Provider IDs
- Duplicated attachments

Tier 4

Expand STP % and ML integration

- Fix technical fallouts
- Attachment classification
- Clinical notes (BERT)
- Historical claims matching
- Real-time feature store

To be explored Medical assessment for systems other than UPM, which decoupling Clawdia from UPM would make possible.

Modernisation

Lift and shift to Docker

Move more than 20 services from IIS on virtual machines to container apps.

Blue / green deployments

Release to an idle copy of the platform, then switch traffic to it, with instant rollback.

Multi-insurance environments

Components tied to an insurer's own repository stay per insurer, as today. Components with no insurer repository are shared across all insurer environments.

Catch 1, 2 and 3

Simplify and modernise the Catch applications.

Refactor the plumbing between components

Decouple Clawdia from UPM

UPM no longer initiates Clawdia.

VetHub / COS polling

Replace polling with events, so components react to changes instead of repeatedly checking for them.

Fix data integrity at the source

Duplicated Service Provider IDs

Fix duplicated vet clinic IDs through the Mastering Data initiative.

Duplicated attachments

Save each attachment once, in either VetHub or COS, not in both.

Expand STP % and ML integration

Fix current technical fallouts

Around 10% of claims fall out for technical reasons. Fix the causes and make the system resilient with auto-recovery.

Use data we currently ignore

- Attachment classification, e.g. a CPL test result.
- Clinical information from clinical notes: BERT model integration.

Historical claims matching

Enhance the easy cohort with historical claims matching techniques.

Real-time feature store

Features are computed as claim data arrives, so models always work from the latest picture of the claim.

Did you know? When the ML team found that the original case-linking strategy didn't work, they switched to a feature very similar to diagnosis lineage and got much better results.

Gateways at a glance

Each gateway stands on its own business drivers and architecture principles, not only on its contribution to STP.

Tier	Gateway	Main drivers	Main principles and policies
1	Lift and shift to Docker	D11, D12, D17	AP2, AP3, AP9, PO6
1	Blue / green deployments	D1, D11, D17	AP3, AP9, PO5, PO6
1	Multi-insurance environments	D9, D11, D13, D17	AP1, AP2, AP5, AP6
1	Catch 1, 2 and 3	D11, D12, D16	AP1, AP3, AP4, PO6
2	Decouple Clawdia from UPM	D9, D10, D16	AP2, AP5, PO2
2	VetHub / COS polling to events	D1, D12	AP1, AP3, AP5, PO6
3	Duplicated Service Provider IDs	D13, D14	AP1, AP5, PO4
3	Duplicated attachments	D13, D16	AP1, AP4, AP5
4	Fix technical fallouts	D16, D17	AP3, AP4, PO6
4	Attachment classification	D16	AP4, PO1, PO1.1
4	Clinical notes extraction (BERT)	D16	AP4, AP10, PO1, PO1.1
4	Historical claims matching	D16, D17	AP1, AP2, PO1
4	Real-time feature store	D1, D12, D16	AP3, PO6
–	Assessment for non-UPM systems (to be explored)	D9, D10	AP2, PO1.1, PO2, PO7

D = driver · AP = principle · PO = policy, from the Business Motivation Model (key at the end)

Lift and shift to Docker

Drivers

D11 Technical debt and key-person risk: ageing IIS VMs depend on a few people.

D12 Scalability demand: 2.5M+ claims a year. Containers scale out; fixed VMs don't.

D17 Resource constraints: changes the hosting, not the code; the cheapest way to retire the VMs.

Principles and policies

AP2 Pragmatism over perfection: a small, reversible step, not a re-architecture.

AP3 Design for failure and resilience: containers restart and isolate faults.

AP9 Automation is engineered: images bring builds and deployments into pipelines.

PO6 Protect published SLOs: no migration ships if it regresses availability or cycle time.

High-level solution

- Containerise the 20+ IIS services as they are, one image per service.
- Host on Azure Container Apps; landing zones and configuration.
- Package the migration into waves, lowest risk first, keeping the VMs for rollback.

Unknowns and decisions

- Which services need Windows containers (.NET Framework)?
- Azure Container Apps or AKS?
- State held on the VMs: sessions, file shares, certificates.

Missing tasks for the proposal

- Inventory of the 20+ services: runtime, dependencies, owner.
- Wave plan with rollback criteria.
- Capacity baseline against 2.5M claims a year. **(what is the right number??)**

Traces to: S3 Engineer for resiliency and scale · T3.2 Identify bottlenecks to 2.5M+ claims · O1.3 Support Trademark 2030 volumes

Blue / green deployments

New

Drivers

D1 Point-of-care claim expectation: real-time claiming leaves no window for release downtime.

D11 Technical debt and key-person risk: manual releases depend on a few people.

D17 Resource constraints: safer, repeatable releases with a small team.

Principles and policies

AP3 Design for failure and resilience: roll back by switching traffic back.

AP9 Automation is engineered: the switch is a pipeline step, not a manual task.

PO5 Right-sized testing: smoke-test the idle stack before it takes traffic.

PO6 Protect published SLOs: releases stop costing availability.

High-level solution

- TBD

Unknowns and decisions

- Existing approach for other containerised apps
- Database changes both stacks can run against (expand and contract).
- In-flight claims and messages during the switch.
- Which component goes first: VetHub, COS or Clawdia?

Missing tasks for the proposal

- Per-layer definition of "down" and its SLOs.
- Release and rollback runbook.
- Schema migration guideline.

Traces to: S3 Engineer for resiliency and scale · T3.6 Always-on availability · T3.4 Real-time view of Service Level Objects · O1.3 Support Trademark 2030 volumes

Multi-insurance environments

Drivers

D9 Revenue diversification: one shared platform is the basis for serving other insurers.

D11 Technical debt and key-person risk: each duplicated environment is more to patch and deploy.

D13 Security and data-integrity exposure: a multi-administrator model needs deliberate separation.

D17 Resource constraints: fewer environments to run with a small team.

Principles and policies

AP1 Simplicity first: remove per-insurer copies of components that only route.

AP2 Pragmatism over perfection: routing-only components first; data owners later.

AP5 Clear coupling: explicit about what is shared and what is per insurer.

AP6 Secure by design: insurer separation is designed in, not bolted on.

High-level solution

- Tied to an insurer's repository (e.g. a UPM instance): stays per insurer.
- Not tied to any insurer's repository: one shared instance.
- Shared components route by insurer ID.

Unknowns and decisions

- Which components only route today?
- How the insurer is identified: route, token or payload.
- Contract or data-residency limits on sharing

Missing tasks for the proposal

- Component-by-insurer matrix, current and target.
- Insurer isolation and access-control design.
- Run-cost comparison.

Traces to: S1 Decouple from insurer · S2 Invest in component-based software · T1.5 Global Administrators model · T3.8 Logical segregation of administrators · O2.3 Common platform for all insurers

Catch 1, 2 and 3

Drivers

D11 Technical debt and key-person risk: CATCH.API on old .NET Framework; Catch 1.0 on old Angular.

D12 Scalability demand: Catch 2.0's concurrency issue worsens as volumes grow.

D16 Automation imperative: Catch is where humans handle exceptions as automation grows.

Principles and policies

AP1 Simplicity first: merging or retiring applications beats maintaining several.

AP3 Design for failure and resilience: degrade gracefully under load.

AP4 Automate first, human by exception: design around exceptions and control points.

PO6 Protect published SLOs: no regression in availability or cycle time.

High-level solution

- merge 1 and 2 into a single application for exception handling, retiring the overlap.
- Fix the Catch 2.0 concurrency issue, e.g. locking on work items.
- upgraded CATCH.API .NET Framework and move Catch 1.0 off old Angular.

Unknowns and decisions

- What each Catch does today, and where they overlap.
- Move Vet clinic registration somewhere else?
- Integrate now or later with a task manager for task assignment?
- Make Catch the quick Fix Bay UI?

Missing tasks for the proposal

- Capability map of Catch 1, 2 and 3 by user and journey.
- Target UI for exceptions and the quick fix bay.
- Migration and retirement plan.
- Formalise dependencies with duplicated Service Provider Id

Traces to: S3 Engineer for resiliency and scale · T3.2 Identify bottlenecks to 2.5M+ claims · T3.5 Design for automation, escalate to humans · O4.3 Humans handle exceptions only

Decouple Clawdia from UPM

Drivers

D10 Growing claims processing without UPM: less reliance on Gratex/UPM legacy code.

D9 Revenue diversification: a Clawdia without UPM can serve other insurers.

D16 Automation imperative: assessment starts earlier, independent of UPM's timing.

Principles and policies

AP5 Clear coupling: replace "UPM initiates Clawdia" with an explicit contract.

AP2 Pragmatism over perfection: decouple step by step, keeping today's flows working.

PO2 Adjudicate independent of policy ownership: assess whoever owns the policy.

High-level solution

- Start Clawdia as soon as a new conversation is submitted
- Correlate conversation id, claim reference number and Workflow Id
- Prepare the data before UPM looks for a medical assessment. **(for other changes)**
- UPM calls Dynamic Clapi which retrieve the workflow id based on the claim reference number

Unknowns and decisions

- Where to keep the correlation (workflow id, conversation id, and claim reference number)

Missing tasks for the proposal

- Current-state sequence of UPM calls into STP. (80% done)
- Target sequence and contract.
- Group this change with other proposed changes

Traces to: S1 Decouple from insurer · T1.1 COS and Clawdia run without UPM · T1.3 Lifecycle-stage lodgement · T6.5 Clawdia independent of policy record · O1.1 Current claim process without UPM

VetHub / COS polling to events

Drivers

D1 Point-of-care claim expectation: polling adds latency against the 30-second cycle time.

D12 Scalability demand: polling load grows with volume, whether or not anything changed.

Principles and policies

AP1 Simplicity first: removing the polling loop beats tuning it.

AP3 Design for failure and resilience: events can be retried and replayed.

AP5 Clear coupling: events make the VetHub–COS dependency explicit.

PO6 Protect published SLOs: the cycle-time SLO measures success.

High-level solution

- VetHub and COS publish events (e.g. claim submitted, assessed) to a broker.
- Consumers subscribe; idempotent handlers with retry and dead-letter.
- Polling stays as a fallback until events are proven.

Unknowns and decisions

- Broker: Azure Service Bus or Event Grid?
- Who owns event schemas and versions?
- Ordering and duplicate-delivery needs per event.

Missing tasks for the proposal

- List of polling loops, intervals and what they check.
- Event catalogue.
- Latency baseline against the 30-second target.

Traces to: S3 Engineer for resiliency and scale · O3.1 Gap Only STP real-time claiming (cycle time ≤ 30 seconds)

Duplicated Service Provider IDs

Drivers

D14 Data inconsistency across systems: several sources of truth for service providers; clinic-ID drift.

D13 Security and data-integrity exposure: heightened by the multi-administrator model.

Principles and policies

PO4 Master for shared Vet IDs: one system of record, a mapping layer, no manual sync.

AP5 Clear coupling: every consumer knows the system of record.

AP1 Simplicity first: retire the manual dual-insert function.

High-level solution

- Fix duplicated vet clinic IDs through the Mastering Data initiative.
- One master; other systems map to it through an anti-corruption layer.
- Retire the manual dual-insert across UPM instances.

Unknowns and decisions

- Which system is the master?
- Merge rules, and claims recorded against old IDs.
- Scope and timing of the Mastering Data initiative.

Missing tasks for the proposal

- Size the duplicates per system.
- Agree interfaces with the Mastering Data team.
- Mapping-layer design and ADR.

Traces to: S1 Decouple from insurer · T1.2 Unique GO Service Provider IDs

Duplicated attachments

Drivers

D13 Security and data-integrity exposure: duplicates undermine trust in claim data.

D16 Automation imperative: duplicates add noise to classification and assessment.

Principles and policies

AP1 Simplicity first: store once at submission rather than deduplicate downstream.

AP5 Clear coupling: one component owns attachments; others hold a reference.

AP4 Automate first, human by exception: clean inputs mean fewer escalations.

High-level solution

- Save each attachment once, in VetHub or COS, not both.
- The other component keeps a reference only.
- A content hash detects resubmitted files.

Unknowns and decisions

- Owner: VetHub or COS?
- How UPM and Catch read attachments today.
- Retention and access rules for the single copy.

Missing tasks for the proposal

- Trace the attachment flow end to end.
- Measure duplicate volume and storage.
- Plan for existing duplicates.

Traces to: S6 Automate lifecycle with data + AI & ML · T6.3 Invest in Clawdia's STP rate

Fix technical fallouts

Drivers

D16 Automation imperative: around 10% of claims fall out for technical reasons.

D17 Resource constraints: fixing known failures is the cheapest STP gain.

Principles and policies

AP3 Design for failure and resilience: recover automatically instead of dropping to manual.

AP4 Automate first, human by exception: humans see business exceptions, not technical ones.

PO6 Protect published SLOs.

High-level solution

- Classify technical fallouts by cause; fix the top causes first.
- Auto-recovery: retries, idempotent steps, replay from the failed step.
- Technical failures go to an ops queue, not to assessors.

Unknowns and decisions

- Breakdown of the ~10% by cause and component.
- Which steps are safe to retry automatically?
- Who owns the ops queue and alerts?

Missing tasks for the proposal

- Fallout taxonomy and dashboard.
- Recovery pattern per failure type.
- Target for automatic recovery time.

Traces to: S3 Engineer for resiliency and scale · T3.5 Design for automation, escalate to humans · O3.1 Gap Only STP real-time claiming · O4.3 Humans handle exceptions only

Attachment classification

Drivers

D16 Automation imperative: evidence assessors read in attachments is invisible to the models.

Principles and policies

AP4 Automate first, human by exception: give automation the evidence a human uses.

PO1 Condition-coded claims from PetSure taxonomy: results map to the taxonomy.

PO1.1 Gap Only IP protection: the taxonomy stays inside the platform boundary.

High-level solution

- Classify each attachment on arrival: invoice, lab result, history, image.
- Extract key results, e.g. a CPL test result, as structured features.
- Store them in the feature store for Clawdia.

Unknowns and decisions

- Build, buy, or use a document-AI service?
- Which attachment types pay off first?
- Accuracy needed before a feature drives automation.

Missing tasks for the proposal

- Labelled sample of attachment types.
- Feature definitions with the ML team.
- Privacy review of attachment content.

Traces to: S6 Automate lifecycle with data + AI & ML · T6.3 Invest in Clawdia's STP rate

Clinical notes extraction (BERT)

Drivers

D16 Automation imperative: notes hold the diagnosis, but models only use embeddings today.

Principles and policies

AP4 Automate first, human by exception: the evidence a human uses.

AP10 Human accountability for AI decisions: extraction informs; it never declines.

PO1 Condition-coded claims from PetSure taxonomy: terms map to the taxonomy.

PO1.1 Gap Only IP protection: the taxonomy stays inside the boundary.

High-level solution

- A BERT model (e.g. PetBERT) extracts body parts, observations and diagnoses.
- Extracted terms map to the PetSure taxonomy.
- Published as features; Clawdia uses them when notes exist.

Unknowns and decisions

- Fine-tuning effort, and who hosts the model.
- Latency and cost per claim.
- Notes that arrive after settlement (T6.4).

Missing tasks for the proposal

- ML team estimate for PetBERT fine-tuning.
- Serving and integration design.
- Evaluation against assessor coding.

Traces to: S6 Automate lifecycle with data + AI & ML · T6.3 Invest in Clawdia's STP rate · T6.4 Claims without clinical notes

Historical claims matching for the easy cohort

Drivers

D16 Automation imperative: proven history can settle case linking without a model.

D17 Resource constraints: rules are cheaper to build, explain and maintain.

Principles and policies

AP1 Simplicity first: where a deterministic rule works, it beats a model.

AP2 Pragmatism over perfection: build on what worked, such as diagnosis lineage.

PO1 Condition-coded claims from PetSure taxonomy.

High-level solution

- Easy cohort rules that match a claim to the pet's history.
- Link cases through diagnosis lineage.
- Option: a graph database of historical claims.

Unknowns and decisions

- Does easy cohort ownership move to the ML team?
- Graph database, or rules over existing data?
- Precision needed before auto-linking.

Missing tasks for the proposal

- Back-test lineage rules on past claims.
- Estimate the STP uplift: [__]%.
- Agree rule ownership and change process.

Traces to: S6 Automate lifecycle with data + AI & ML · T6.2 Insured asset database · T6.7 Deterministic eligibility gate · O4.2 Auto-assessment based on condition coding

Real-time feature store

Drivers

D1 Point-of-care claim expectation: features must be ready when the claim is.

D12 Scalability demand: computing on arrival spreads load away from assessment time.

D16 Automation imperative.

Principles and policies

AP3 Design for failure and resilience: precomputed features don't wait on a slow source.

PO6 Protect published SLOs: supports the cycle-time SLO.

High-level solution

- Compute features from VetHub and COS events as data arrives.
- Online store for assessment, offline store for training, same definitions.
- Depends on polling to events.

Unknowns and decisions

- Product: Azure ML feature store, Feast or custom?
- Freshness needed per feature.
- Owner: platform team or ML team?

Missing tasks for the proposal

- Feature inventory with freshness needs.
- Event-to-feature pipeline design.
- Cost estimate.

Traces to: S6 Automate lifecycle with data + AI & ML · O3.1 Gap Only STP real-time claiming (cycle time ≤ 30 seconds)

Medical assessment for non-UPM systems

To be explored

Drivers

D9 Revenue diversification: sell claims services to other insurers.

D10 Growing claims processing without UPM.

Principles and policies

AP2 Pragmatism over perfection: validate demand before building (YAGNI).

PO2 Adjudicate independent of policy ownership.

PO1.1 Gap Only IP protection: the taxonomy stays inside our boundary.

PO7 Reserve the fast path for PetSure: external insurers on a slower tier.

High-level solution

- Assessment exposed through a platform-agnostic API.
- External insurers on a slower tier, e.g. a 10-minute SLA.
- Results returned without exposing the taxonomy.

Unknowns and decisions

- Is there demand, and from whom?
- Commercial model and data ownership (PO8).
- What partners need back: codes, a decision or both.

Missing tasks for the proposal

- Validate demand with the business first.
- Draft API contract.
- Sequence it after decoupling Clawdia from UPM.

Traces to: S1 Decouple from insurer · T1.4 Platform-agnostic APIs · T6.5 Clawdia independent of policy record · O1.2 At least one new insurer on Gap Only

Clawdia starts with the conversation

Clawdia, the medical assessment orchestration platform, starts as soon as a new conversation is submitted.

01 · Clawdia

Conversation submitted

Clawdia starts work straight away.

02 · Clawdia

Classified and processed

Claim data is ready before UPM lodges the claim.

03 · UPM asks

Assessment requested

Clawdia already knows most of the answer, if not all of it.

UPM still owns the concept of **logical claims**.

Today we have only two strategies

Easy cohort

Business rules.

GP model inference

The claim goes in as it is, and comes back with ML predictions and error codes.

```
E101 InvalidInputError
field <TStarCToken> is missing
```

New AI-driven strategies can be added as the technology matures, without redesigning the platform.

The GP model output, changing in 12/2026

MLPrediction, renamed from GPModelPredictions. Every field becomes nullable.

Field	Type	Change (12/2026)	Provided by
DstarPrimeToken	String	Required → nullable	ML: GP model or diagnosis token
Confidence	Number	Required → nullable	ML: GP model or diagnosis token
Reason	String	Required → nullable	ML: GP model or diagnosis token
IsMultiDiagnosis	Boolean	Required → nullable	ML: GP model or diagnosis token
MultiDiagnosisConfidence	Number	Required → nullable	ML: GP model or diagnosis token
CaseCardCode	String	Renamed from CaseCard; required → nullable	Condition Linking model
CaseCardName	String	New	Condition Linking model
CaseMatchConfidence	Number	New	Condition Linking model
CaseReason	String	New	Condition Linking model
TotalNoCaseCards	Integer	New	Condition Linking model

Shifting where AI helps

> *Use AI to shrink manual effort, not only to push up STP.*

~25%

STP this year, from our current strategies

~70%

STP possible later

~30%

still assessed manually

AI's job: reduce the effort of manual assessment, until diagnosis families that used to need a person can be automated.

Model it on what assessors actually do

ASSESSORS DON'T

Diagnose the animal.

The vet has already done that.

ASSESSORS DO THREE THINGS

1. Check that the claim data, including consultation notes and treatments, is legitimate.
2. Rule out diagnosis families that don't fit.
3. Answer specific questions to find the right code within the families that remain.

> The diagnosis is already in the consultation notes or attachments. Assessors apply veterinary logic to turn it into accurate diagnosis codes.

What a state-of-the-art AI agent can already do

01

Clawdia collects all the claim data into a queryable context. A simple JSON object is enough.

02

Clawdia rules out diagnosis families that don't fit the claim.

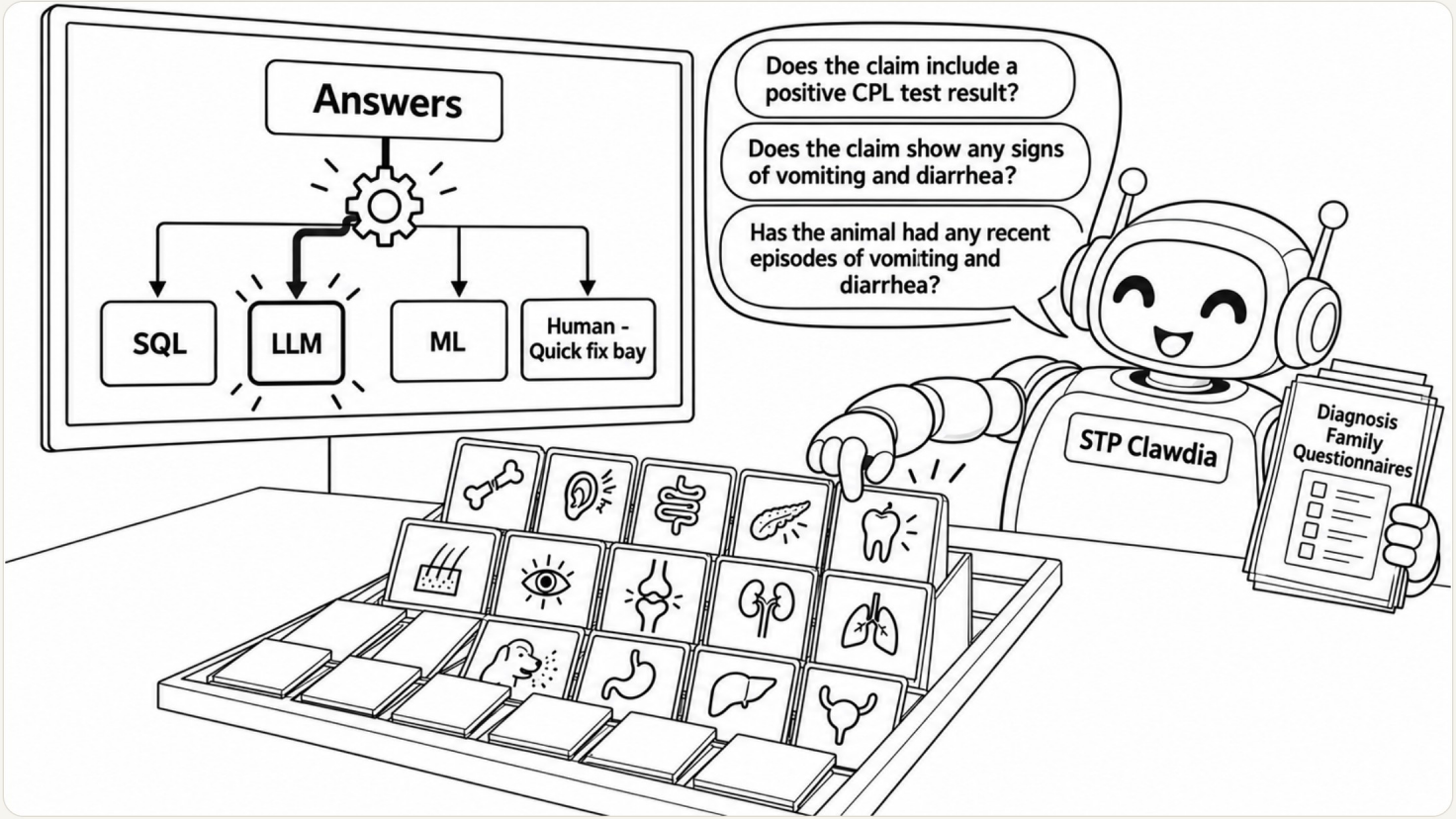
03

Clawdia generates a list of questions for the agent.

04

Clawdia organises a game of *Guess Who?* with the agents.

How a game of Guess Who works



01 · The board

Each tile is a diagnosis family. Each answer flips down the families that don't fit.

02 · The questions

Clawdia asks them from configured, deterministic diagnosis-family questionnaires.

03 · The answers

Each comes from the source that fits: SQL, an LLM, an ML model, or a person in the quick fix bay.

How the game ends, and how it gets better

Confident answer

The diagnosis is assigned automatically.

Not confident enough

A quick fix bay reviews the remaining suspect diagnoses.

Each human answer becomes training data. Over time, the system learns to identify the diagnosis confidently on its own.

NEXT STEPS

The ask

01 Check that we're aligned on the vision.

02 Check which gateways are relevant, and whether there are others to prioritise.

03 Organise a team effort to push our agenda with the business.

APPENDIX

Reference key: Business Motivation Model

Drivers

D1 Point-of-care claim expectation
D9 Revenue diversification
D10 Growing claims processing without UPM
D11 Technical debt and key-person risk
D12 Scalability demand
D13 Security and data-integrity exposure
D14 Data inconsistency across systems
D16 Automation imperative
D17 Resource constraints

Architecture principles

AP1 Simplicity first
AP2 Pragmatism over perfection
AP3 Design for failure and resilience
AP4 Automate first, human by exception
AP5 Clear coupling
AP6 Secure by design
AP9 Automation is engineered
AP10 Human accountability for AI decisions

Policies

PO1 Condition-coded claims from PetSure taxonomy
PO1.1 Gap Only IP protection
PO2 Adjudicate independent of policy ownership
PO4 Master for shared Vet IDs
PO5 Right-sized testing
PO6 Protect published SLOs
PO7 Reserve the fast path for PetSure

Source: 90% STP Architecture Vision, Business Motivation Model (draft, 3 August 2026)